

Delve debugger

이준서



목차

- Debugging 소개
 - Debugging 중요성
 - Logging 통한 Debugging
 - Debugger 활용
- Delve
 - delve 기능(attach, debug, exec)
 - Debugging 예시
 - gdlv

Debugging 소개



Debugging 정의 및 중요성

- Debugging 정의
 - 컴퓨터 프로그램 상의 오류(버그)를 찾아내어 바로잡는 과정
- Debugging 중요성
 - Debugging 과정을 통해 버그를 찾고 수정할 수 있음
 - 버그의 원인을 파악하고 해결 가능
 - 코드가 예상대로 동작하지 않을 때 문제의 근본 원인을 찾아냄

Debugging 기법

- 로그를 활용한 Debugging
- Debugger를 통한 Debugging
 - 중단점 설정 및 변수 값 확인
 - 스택 트레이스 분석: 프로그램 실행 중에 어떤 함수 호출되었는지 보여줌

Logging 통한 Debugging

➤ Logging의 정의

- 프로그램이 실행되는 동안 중간 결과나 오류 등 발생하는 이벤트의 로그를 유지하는 행위
- 로그: 컴퓨터 시스템 활동의 정보를 담고 있는 파일이나 기록

➤ 중요 이벤트와 상태 변화를 기록하여 문제 발생 시 로그 분석해서 원인 파악

➤ Logging의 장점

- 로그 레벨 지원
 - 로그의 중요도에 따라 로그 레벨 (DEBUG, INFO, WARN, ERROR, FATAL) 설정 가능
- 출력 포맷 제어
 - 로그 메시지의 포맷을 지정할 수 있는 기능을 제공
 - 타임스탬프, 로그 레벨, 파일명, 라인 번호 등
- 로그 출력 경로 설정
 - 파일, 콘솔, 네트워크 등 다양한 경로로 출력 가능

Debugger 정의

- Debugging 도구로서 프로그램을 테스트하고 Debug하는데 쓰이는 프로그램
 - go 언어용 공식 Debugger: delve

Debugger 기능

➤ 실시간 상태 검사

- 코드 실행 중에 변수의 현재 값, 메모리 상태, 호출 스택 등을 실시간으로 확인 가능
- 문제 발생 시점의 정확한 프로그램 상태를 알 수 있음
- 로깅은 사후 분석을 위한 것으로 로그 메시지를 통해 간접적으로 상태 파악

➤ 브레이크포인트 설정과 단계별 실행

- 코드를 한 줄 씩 또는 한 함수 씩 실행하여 문제를 점진적으로 파악 가능
- 문제 발생 지점을 정확하게 파악 가능하고 조건부 브레이크포인트도 사용가능
- 로깅은 연속적인 실행 흐름을 중단하지 않고 메시지를 남기므로, 세밀한 단계별 분석이 어려움
- 여러 지점에 로그를 삽입하면 코드가 지저분해 지고 분석 또한 어려워지며 성능 저하 가능성 있음

➤ 호출 스택 추적

- 함수의 흐름을 실시간으로 확인할 수 있음

Debugging 방법

- Logging만을 사용해도 문제 없지만 Debugger를 사용할 때 더 효율적
 - 문제 지점을 찾을 수 있지만 Debugger에 비해 효율성이 떨어지기 때문에 사용 추천
- Debugger만을 사용해도 비효율적임
 - Logging 설정하여 주요 이벤트 및 상태 변화 기록해 두면, 발견하지 못한 문제를 로그를 통해 추가로 발견할 수 있음
 - 간헐적 문제나 복잡한 버그의 경우, 문제 발생 패턴과 빈도를 추적하기에는 Logging이 효율적
 - 프로덕션 환경에서 문제 발생 경우, Debugger 사용 불가능하거나 제한적임
- Debugger와 Logging 함께 사용
 - 로그 기반으로 문제 재현하고 Debugger 사용하여 수정 작업 진행

Delve 소개



Delve 시작

- Delve 정의
 - go언어를 위한 Debugger
- 설치
 - go install github.com/go-delve/delve/cmd/dlv@latest
- Debugging
 - Debugging하려고 하는 위치에서 dlv debug를 command line에 입력하면 시작
 - 다른 위치에서도 dlv debug “directory”로 시작 가능
 - “--”로 flags 전달

Delve 기능

- **dlv debug**
 - 메인 패키지를 컴파일하고 Debugging 시작
- **dlv exec**
 - 컴파일 된 binary파일 실행시키고 Debugging 시작
- **dlv test**
 - test binary를 컴파일하고 Debugging 시작
- **dlv attach**
 - running 중인 process에 연결하고 Debugging 시작
- **dlv connect**
 - headless Debug 서버에 터미널 클라이언트를 사용해서 연결

debug 실행 명령어

- **continue (c)**
 - 중단점 또는 프로그램 종료까지 실행
- **next (n)**
 - 다음 라인 실행
- **step (s)**
 - 다음 스텝 실행
 - next와의 차이점: 다음 줄에 함수 호출이 있으면 안으로 들어감
- **step-instruction (si)**
 - CPU instruction 단위로 실행
 - 어셈블리어로 디버깅
- **stepout (so)**
 - 현재 함수의 실행을 완료하고 그 함수를 호출한 위치로 돌아감

중단점 생성

➤ break (b)

- 중단점 설정
- 파일과 라인 번호로 설정
 - `break main.go:10`
- 함수 이름으로 설정
 - `break main.myFunction`
- **offset**으로 설정
 - 현재 위치 line에서 +/-로 중단점 설정 가능 `b +<offset>`

➤ breakpoints

- 현재 활성화된 중단점들 출력

➤ condition (cond)

- 중단점의 조건부를 생성
- `cond -hitcount <breakpoint name or id> <operator> <argument>` 로 중단점의 hit count 조건부 생성 가능
- `cond -hitcount 3 == 4`

중단점 삭제

- **clear**
 - 특정 중단점 삭제
- **clearall**
 - 모든 중단점 삭제
- **toggle**
 - 특정 중단점을 enable/disable 시킴

tracepoint 생성

➤ trace (t)

- tracepoint에 도달하면 실행 중단하지 않고 그 지점의 정보를 기록
- trace [name] [locspec]

변수 보기

- args
 - 함수 arguments 출력
- display
 - **display -a [%format] <expression>**
 - 프로그램 중단마다 expression의 값을 출력하는 list에 추가
 - **display -d <number>**
 - list에 있는 display expression 삭제
- print (p)
 - expression의 결과 출력
- locals
 - 지역변수들 값 출력
- set
 - 변수 값 변경
- whatis: expression의 type 출력

- All (binary and unary) on basic types except <-, ++ and --
- Comparison operators on any type
- Type casts between numeric types
- Type casts of integer constants into any pointer type and vice versa
- Type casts between string, []byte and []rune
- Struct member access (i.e. `somevar.memberfield`)
- Slicing and indexing operators on arrays, slices and strings
- Map access
- Pointer dereference
- Calls to builtin functions: `cap` , `len` , `complex` , `imag` and `real`
- Type assertion on interface variables (i.e. `somevar.(concretetype)`)

<delve에서의 expression>

메모리 보기

- **examinemem (x)**
 - 주소값에 있는 실제 data 값 제공
- **regs**
 - register들 제공
- **list (ls)**
 - [goroutine <n>] [frame <m>] list [<locs-spec>]
 - 소스코드 보여줌

thread와 goroutines listing 및 switching

- **goroutine (gr)**
 - **현재 goroutine 보여줌**
 - 어떤 Thread에 있고, User, go, start등의 정보 담겨져있음
 - **gr <id>**
 - 다른 goroutine으로 바꿈
 - **gr <id> <command>**
 - id에 맞는 goroutine에서 command 실행(print나 set 등)
- **goroutines (grs)**
 - 프로그램 goroutines 열거
 - flags들을 통해서 원하는 goroutines 보여줄 수 있음
- **thread (tr)**
 - 특정 thread로 변경
- **threads**
 - 모든 traced thread의 정보 출력

호출 스택 보기

- **stack (bt)**
 - 호출된 스택 출력
- **up**
 - 현재 frame의 up으로 이동
- **down**
 - 현재 frame의 down으로 이동
- **frame**
 - frame <m> (<command>)
 - current frame을 설정하거나 다른 frame에서 command 실행
 - print나 set같은 command 실행 가능

configuration 관리

➤ config

- `config -list`: 모든 configuration parameters를 보여줌
- `max-string-len`이 초기에 not defined로 되어있는데 설정해주지 않으면 값이 생략되서 나오는 issue가 있음
 - 긴 string 사용한다면 1000정도로 설정해주면 issue 해결 가능
- `source-list`의 line, keyword, string, number 등 color 설정 가능
- `config -save`: 설정한 configuration을 disk에 저장

예제 설명

➤ Sort

- array를 정렬하는 알고리즘
- selection, insertion, merge, quick sort 각각을 goroutine으로 실행

```
var wg sync.WaitGroup
wg.Add(4)
original := RandArray(25)
arr := make([]int, len(original))
for i := 0; i < 4; i++ {
    go func(id int) {
        defer wg.Done()
        copy(arr, original)
        if i == 0 {
            SelectionSort(arr)
            fmt.Println("SelectionSort:", arr)
        } else if i == 1 {
            InsertionSort(arr)
            fmt.Println("InsertionSort:", arr)
        } else if i == 2 {
            MergeSort(arr, 0, len(arr)-1)
            fmt.Println("MergeSort:", arr)
        } else {
            QuickSort(arr)
            fmt.Println("QuickSort:", arr)
        }
    }(i)
}
wg.Wait()
fmt.Println("All goroutines finished")
```

<예제 source code>

예제 중단점 설정

- 각 함수 시작마다 중단점 설정

```
(dlv) b main.main
Breakpoint 1 set at 0xcc4053 for main.main() C:/Users/이준서/go/osdc_2024/dlv_test/sort/main.go:23
(dlv) b SelectionSort
Breakpoint 2 set at 0xcc5644 for main.SelectionSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/selectionSort.go:3
(dlv) b InsertionSort
Breakpoint 3 set at 0xcc3d44 for main.InsertionSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/insertionSort.go:3
(dlv) b MergeSort
Breakpoint 4 set at 0xcc4f0e for main.MergeSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/mergeSort.go:31
(dlv) b QuickSort
Breakpoint 5 set at 0xcc5076 for main.QuickSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/quickSort.go:5
(dlv) █
```

<중단점 설정 예시>

예제 스택 보기

- 몇번의 continue 이후 스택 확인
- quickSort는 재귀함수를 사용하기 때문에 스택에 함수가 쌓임
- 각 스택은 up/down으로 이동할 수 있고 현재 0번 스택에 있어도 1번, 2번 스택의 변수 값을 frame 1 print arr 명령어로 알 수 있음

```
(dlv) bt
0 0x00000000cc5076 in main.QuickSort
   at C:/Users/이준서/go/osdc_2024/dlv_test/sort/quickSort.go:5
1 0x00000000cc53b1 in main.QuickSort
   at C:/Users/이준서/go/osdc_2024/dlv_test/sort/quickSort.go:22
2 0x00000000cc53e6 in main.QuickSort
   at C:/Users/이준서/go/osdc_2024/dlv_test/sort/quickSort.go:23
3 0x00000000cc4907 in main.main.func1
   at C:/Users/이준서/go/osdc_2024/dlv_test/sort/main.go:63
4 0x00000000cc4391 in main.main.gowrap1
   at C:/Users/이준서/go/osdc_2024/dlv_test/sort/main.go:66
5 0x00000000c7da81 in runtime.goexit
   at C:/Program Files/Go/src/runtime/asm_amd64.s:1695
```

<예제 stack명령어(bt)명령어 >

```
(dlv) frame 1 print arr
[]int len: 19, cap: 25, [17,9,12,10,18,5,8,23,23,22,13,7,9,9,7,13,22,20,13]
(dlv) frame 2 print arr
[]int len: 25, cap: 25, [4,17,9,12,10,18,4,5,8,23,23,3,22,13,7,9,3,9,7,13,0,22,20,13,3]
(dlv) print arr
[]int len: 12, cap: 19, [9,12,10,5,8,13,7,9,9,7,13,13]
```

<frame 명령어 실행 예시>

display 예제

➤ display 명령어가 array에 대해서 [...]으로 나옴

- config로 나오게 하는 방법 찾아보았지만
방법 없음
- print 를 통해서 자세한 값 볼 수 밖에 없음

```
(dlv) display -a arr
0: arr = []int len: 25, cap: 25, [...]
(dlv) display -a i
1: i = error could not find symbol value for i
```

<display 명령어 예시>

```
(dlv) c
> main.QuickSort() C:/Users/이준서/go/osdc_2024/dlv_test/
1: package main
2:
3: // divide and conquer
4:
=> 5: func QuickSort(arr []int) []int {
6:     if len(arr) <= 1 {
7:         return arr
8:     }
9:     pivot := arr[0]
10:    leftArr := make([]int, 0, len(arr))
0: arr = []int len: 4, cap: 12, [...]
1: i = error could not find symbol value for i
(dlv) c
> main.QuickSort() C:/Users/이준서/go/osdc_2024/dlv_test/
1: package main
2:
3: // divide and conquer
4:
=> 5: func QuickSort(arr []int) []int {
6:     if len(arr) <= 1 {
7:         return arr
8:     }
9:     pivot := arr[0]
10:    leftArr := make([]int, 0, len(arr))
0: arr = []int len: 0, cap: 4, []
1: i = error could not find symbol value for i
```

<중단마다 display>

goroutines 예제

- 예제 코드는 각 정렬을 다른 goroutine으로 실행
- gr 명령어로 다른 goroutine 으로 변경하고 Debugging 가능
- 각 goroutine마다 다른 stack 가지고 있음

```
(dlv) grs
Goroutine 1 - User: C:/Program Files/Go/src/runtime/sema.go:62 sync.runtime_Semacquire (0xc79c85) [semacquire]
Goroutine 2 - User: C:/Program Files/Go/src/runtime/proc.go:403 runtime.gopark (0xc5107c) [force gc (idle)]
Goroutine 3 - User: C:/Program Files/Go/src/runtime/proc.go:403 runtime.gopark (0xc5107c) [GC sweep wait]
Goroutine 4 - User: C:/Program Files/Go/src/runtime/proc.go:403 runtime.gopark (0xc5107c) [GC scavenge wait]
Goroutine 5 - User: C:/Program Files/Go/src/runtime/proc.go:403 runtime.gopark (0xc5107c) [finalizer wait]
Goroutine 6 - User: C:/Users/이준서/go/osdc_2024/dlv_test/sort/main.go:50 main.main.gowrap1 (0xcc4360)
* Goroutine 7 - User: C:/Users/이준서/go/osdc_2024/dlv_test/sort/insertionSort.go:3 main.InsertionSort (0xcc3d44) (thread 15636)
Goroutine 8 - User: C:/Users/이준서/go/osdc_2024/dlv_test/sort/mergeSort.go:31 main.MergeSort (0xcc4f0e) (thread 25372)
Goroutine 9 - User: C:/Users/이준서/go/osdc_2024/dlv_test/sort/quickSort.go:5 main.QuickSort (0xcc5617) (thread 20544)
```

<grs 명령어 실행>

```
(dlv) gr 9
Switched from 8 to 9 (thread 20544)
(dlv) ls
> main.MergeSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/m
> main.InsertionSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort
3d44)
> main.QuickSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/q
1: package main
2:
3: // divide and conquer
4:
=> 5: func QuickSort(arr []int) []int {
6:     if len(arr) <= 1 {
7:         return arr
8:     }
9:     pivot := arr[0]
10:    leftArr := make([]int, 0, len(arr))
(dlv) print arr
[]int len: 3, cap: 4, [8,7,7]
(dlv) gr 8
Switched from 9 to 8 (thread 25372)
(dlv) ls
> main.InsertionSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort
3d44)
> main.MergeSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/m
26:         k++
27:     }
28: }
29: return sorted
30: }
=> 31: func MergeSort(arr []int, left, right int) []int {
32:     var mid int
33:     if left < right {
34:         mid = (left + right) / 2
35:         arr = MergeSort(arr, left, mid)
36:         arr = MergeSort(arr, mid+1, right)
(dlv) b mergeSort.go:34
Breakpoint 7 set at 0xcc4f5a for main.MergeSort() C:/Users/이준
(dlv) c
> main.MergeSort() C:/Users/이준서/go/osdc_2024/dlv_test/sort/m
29:         return sorted
30: }
31: func MergeSort(arr []int, left, right int) []int {
```

<gr 명령어로 debugging>

gdlv

- GUI frontend for Delve
- 오픈소스로 쉽게 설치 가능
- 장/단점
 - 장점: delve보다 가독성 좋은 디버깅 가능
 - stack, goroutine 등 한눈에 보여주고 전체 변수값들 볼 수 있어서 편함
 - 단점: bamboo에서 config파일 읽어오지 못하는 문제 발견하였음
 - 만약 issue들 flag를 통해 해결하는 방법 찾는다면 유의미하게 사용될 수 있음

gdlv 예시

```

Listing
C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go
3 // divide and conquer
4
5 func QuickSort(arr []int) []int {
6     if len(arr) <= 1 {
7         return arr
8     }
9     pivot := arr[0]
10    leftArr := make([]int, 0, len(arr))
11    rightArr := make([]int, 0, len(arr))
12    midArr := make([]int, 0, len(arr))
13    for _, i := range arr {
14        if i < pivot {
15            leftArr = append(leftArr, i)
16        } else if i == pivot {
17            midArr = append(midArr, i)
18        } else {
19            rightArr = append(rightArr, i)
20        }
21    }
22    leftArr = QuickSort(leftArr)
23    rightArr = QuickSort(rightArr)
24
25    leftArr = append(leftArr, midArr...)
26    leftArr = append(leftArr, rightArr...)
27
28    return leftArr
29 }

```

```

Command
> Breakpoint / set at 0x6c4ba/ for main.main.func1() C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:5/
> Breakpoint 8 set at 0x6c47c7 for main.main.func1() C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:60
> Breakpoint 9 set at 0x6c48ea for main.main.func1() C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:63
goroutine 1:0> c
> main.main.func1() C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:54 (hits goroutine(6):1 total:1) (PC: 0x6c4585)
goroutine 6:0> s
> main.SelectionSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/selectionSort.go:3 (PC 0x6c5644)
goroutine 6:0> c
> main.main.func1() C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:63 (hits goroutine(9):1 total:1) (PC: 0x6c48ea)
goroutine 9:0> s
> main.QuickSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5 (PC 0x6c5076)
goroutine 9:0> c
> main.QuickSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:22 (hits goroutine(9):1 total:1) (PC: 0x6c5394)
goroutine 9:0> c
> main.main.func1() C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:57 (hits goroutine(7):1 total:1) (PC: 0x6c46a7)
goroutine 7:0> c
> main.QuickSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:22 (hits goroutine(9):2 total:2) (PC: 0x6c5394)
goroutine 9:0> c
> main.QuickSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:22 (hits goroutine(9):3 total:3) (PC: 0x6c5394)
goroutine 9:0> c
> main.QuickSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:22 (hits goroutine(9):4 total:4) (PC: 0x6c5394)
goroutine 9:0> c
> main.QuickSort() C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:22 (hits goroutine(9):5 total:5) (PC: 0x6c5394)
goroutine 9:0>

```

```

Stacktrace
depth: 50
Full names
Mode: Normal
0 0x6c5394 main.QuickSort
-1880 at C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5
1 0x6c53e6 main.QuickSort
-1544 at C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5
2 0x6c53e6 main.QuickSort
-1208 at C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5
3 0x6c53e6 main.QuickSort
-872 at C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5
4 0x6c53e6 main.QuickSort
-536 at C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5
5 0x6c4907 main.main.func1
-72 at C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:63
6 0x6c4391 main.main.gowrap1
-32 at C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:66
7 0x67da81 runtime.goexit
-24 at C:/Program Files/Go/src/runtime/asm_amd64.s:1695
Created by Goroutine 1
0 0x67ba68 runtime.systemstack switch

```

```

Goroutines
Limit: 100
User location
Add rule...
5 runtime.gopark
at C:/Program Files/Go/src/runtime/proc.go:403 [finalize]
6 18496 sync.(*Pool).pinSlow
at C:/Program Files/Go/src/sync/pool.go:234
7 12244 sync.runtime_doSpin
at C:/Program Files/Go/src/runtime/proc.go:7064
8 main.main.gowrap1
at C:/Users/0000/go/osdc_2024/dlv_test/sort/main.go:50
9 24556 main.QuickSort
at C:/Users/0000/go/osdc_2024/dlv_test/sort/quickSort.go:5

```

```

Variables
Filter:
Full Types Address
Local variables and arguments
arr []int = [6,6,7,7,7,7,8,11,14,14,15,16,17,17,17,19,...+5 more]
~r0 []int = nil
pivot int = 6
leftArr []int = []
rightArr []int = [7,7,7,7,8,11,14,14,15,16,17,17,17,19,19,22,...+3 more]
midArr []int = [6,6]

```

Editor plugins

- delve를 사용한 여러 editor 존재함
 - dlv dap을 통한
 - Atom, Goland, IntelliJ, Vim, VSCode 등등
 - <https://github.com/go-delve/delve/blob/master/Documentation/EditorIntegration.md>